Exercise Sheet 3

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 20.4.2026 Deadline: 27.4.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 3.1 (*Pipelining for RISC-V*).

Consider a classic RISC pipeline consisting of the following five stages:

i) `IF` Instruction Fetch

ii) `ID` Instruction Decode

iii) `EX` Execute

iv) `MEM` Memory Access

v) `WB` Write Back

Assume that each stage requires exactly one time unit that the following sequence of instructions is to be executed on the considered processor:

```
ADD R1, R2, R3    // R1 = R2 + R3
SUB R4, R1, R5    // R4 = R1 - R5
LW  R6, 100(R4)   // R6 = Memory[100 + R4]
OR  R7, R8, R9    // R7 = R8 OR R9 (logical)
```

a) Draw a timing diagram (time on the x-axis, instructions on the y-axis) for the case where no forwarding/data bypassing is supported. Namely, a register can only be read in the `ID` stage after it has been completely written in the `WB` stage of a previous instruction. How many time units $T_{\text{NoForward}}$ are required until the last instruction completes the `WB` stage?

b) Redraw the diagram, this time assuming full forwarding support, i.e., resulting data of an operation can be passed directly from the `EX` or `MEM` stage to the `EX` stage of a subsequent instruction. How many time units $T_{\text{FullForward}}$ are required now?

c) Calculate the speedup achieved by pipelining for the 4 instructions compared to the time $T_{\text{Sequential}}$ of a strictly sequential execution, where one instruction must finish completely before the next begins. Use the result $T_{\text{FullForward}}$ from second task with full forwarding support to determine the speed-up

$$\text{Speedup} = \frac{T_{\text{Sequential}}}{T_{\text{FullForward}}}.$$

 © Advanced Computing 2026

Exercise 3.2 (*Practical Investigation of Pipeline Effects*).

Analyze the impact of instruction dependencies on the execution performance of a code. Therefore, consider a code with data dependencies as baseline, for example, the following C-code:

```
volatile int a = 0, b = 0, c = 0, d = 0, x = 0, y = 0, z = 0;

int baseline_version() {
    int result = 0;
    for (int i = 0; i < N; i++) {
        x = a + b;
        y = x + c;
        z = y + d;
        result += z;
    }
    return result;
}
```

a) Create a modified version of the function performing an equivalent computational workload but allowing the pipeline to execute instructions in parallel, i.e., ensure that the operations/variables (per iteration) are independent.

b) Write a C-code that measures for a varying number of iterations $N \in \mathbb{N}$ the two run-times T_{dep} and T_{ind} for the the baseline function and the modified function with independent operations, respectively.

c) Compile the code with both functions with the compile options `-O0` and `-O2`.

d) Measure the average execution times on Raspberry Pi hardware and calculate the speedup

$$S = T_{\text{dep}}/T_{\text{ind}}.$$

e) Explain the performance differences regarding pipeline utilization, compiler optimization, and hardware features.

f) (Optional) Inspect the generated assembly codes using the compile option `-S`.

 © Advanced Computing 2026